

Containerisierung mit Docker - Installation und erste Schritte

1. Einführung

Docker ist ein modernes Werkzeug, mit dem Softwareentwickler Anwendungen in sogenannten **Containern** ausführen können. Ein Container ist eine **selbstständige**, vom **Host-Betriebssystem isolierte** Einheit, welche alles enthält was man braucht, um eine Anwendung auszuführen: Code, Bibliotheken, Konfigurationsdateien und Laufzeitumgebungen. Dadurch wird die „**Works-on-my-machine**“ Problematik gelöst, bei der eine Anwendung bei einem Entwickler lokal funktioniert, aber bei einem anderen Entwickler oder in der Produktionsumgebung Bugs aufwirft. Denn eine containerisierte Anwendung verhält sich **auf jedem Computer gleich**, da der Container vom Host-System isoliert ist. Das folgende Tutorial führt dich durch die Installation und die ersten Schritte mit Docker.

2. Installation

2.1 Installation auf Windows-Geräten

Lade dir zu erst den passenden Installer für deine Systemarchitektur (AMD64 oder ARM64) herunter und starte ihn: <https://www.docker.com/products/docker-desktop/>

Welche Systemarchitektur du hast, findest du unter *Einstellungen > System > Info > Systemtyp*. Falls dort „x64-basierter Prozessor“ steht wähle AMD64, bei „ARM-basierter Prozessor“ wähle ARM64. 32-Bit Systeme werden nicht unterstützt.

WICHTIG: Für Windows-Geräte ist es erforderlich das „Windows Subsystem for Linux“ (WSL2) zu installieren, weil Docker-Container Linux-basiert laufen. Stelle dafür sicher, dass während der Installation die Option „*Use the WSL 2 based engine*“ ausgewählt ist, falls sie angezeigt wird. Bei älteren Windows-Geräten kann es erforderlich sein, WSL 2 manuell zu installieren, das funktioniert wie folgt:

- 1) Öffne ein Terminal (z.B. cmd oder PowerShell) und führe den Befehl **ws1 --install** aus.
- 2) Suche in der Windows-Suchleiste nach „*Windows-Features aktivieren und deaktivieren*“ und setze in der Liste den Haken beim Eintrag „*Windows-Subsystem für Linux*“.
- 3) Drücke OK und schließe das Fenster, WSL sollte nun startklar sein.

Docker Desktop sollte jetzt vollständig installiert sein. Falls es jetzt noch Probleme gibt, liegt das häufig daran, dass Hardware-Virtualisierung im BIOS ausgeschaltet ist. Wie du das ändern kannst, erfährst du hier: [Aktivieren der Virtualisierung unter Windows](#)

2.2 Installation auf Mac-Geräten

Lade dir zu erst den passenden Installer für dein System (Apple Chip oder Intel Chip) herunter und starte ihn: <https://www.docker.com/products/docker-desktop/>

Um herauszufinden welcher Chip bei dir verbaut klicke links oben auf das Apple-Symbol, dann auf „Über diesen Mac“. Dort findest du unter der Modellbezeichnung Informationen zum Chip.

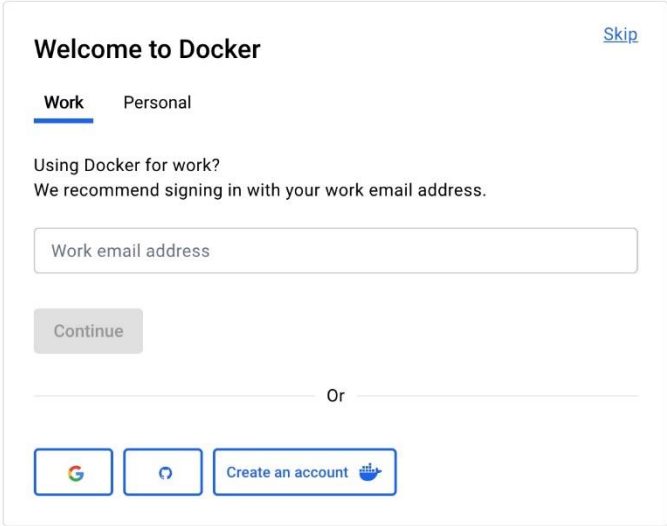
Öffne den Controller mit einem Doppelklick. Ziehe danach das Docker-Icon per Drag and Drop in den Programme-Ordner. Docker wird daraufhin unter / Programme / Docker.app installiert. Du kannst Docker jetzt starten, indem du zu diesem Pfad navigierst und das Docker-Icon doppelt anklickst. Daraufhin startet ein Installer, in welchem du am besten die empfohlenen Standardeinstellungen auswählst.

2.3 Installation auf Linux-Geräten

Da wir nicht auf jede Linux-Distribution gesondert eingehen konnten und es dazu schon einen sehr guten Installationsguide auf der offiziellen Website von Docker gibt, folge am besten dieser Anleitung: <https://docs.docker.com/desktop/setup/install/linux/>

3. Erste Schritte

Nachdem du Docker jetzt installiert hast, starte zuerst die Desktop App. Der Initiale Login-Bildschirm sollte ungefähr so aussehen:



Welcome to Docker [Skip](#)

Work Personal

Using Docker for work?
We recommend signing in with your work email address.

Work email address

Continue

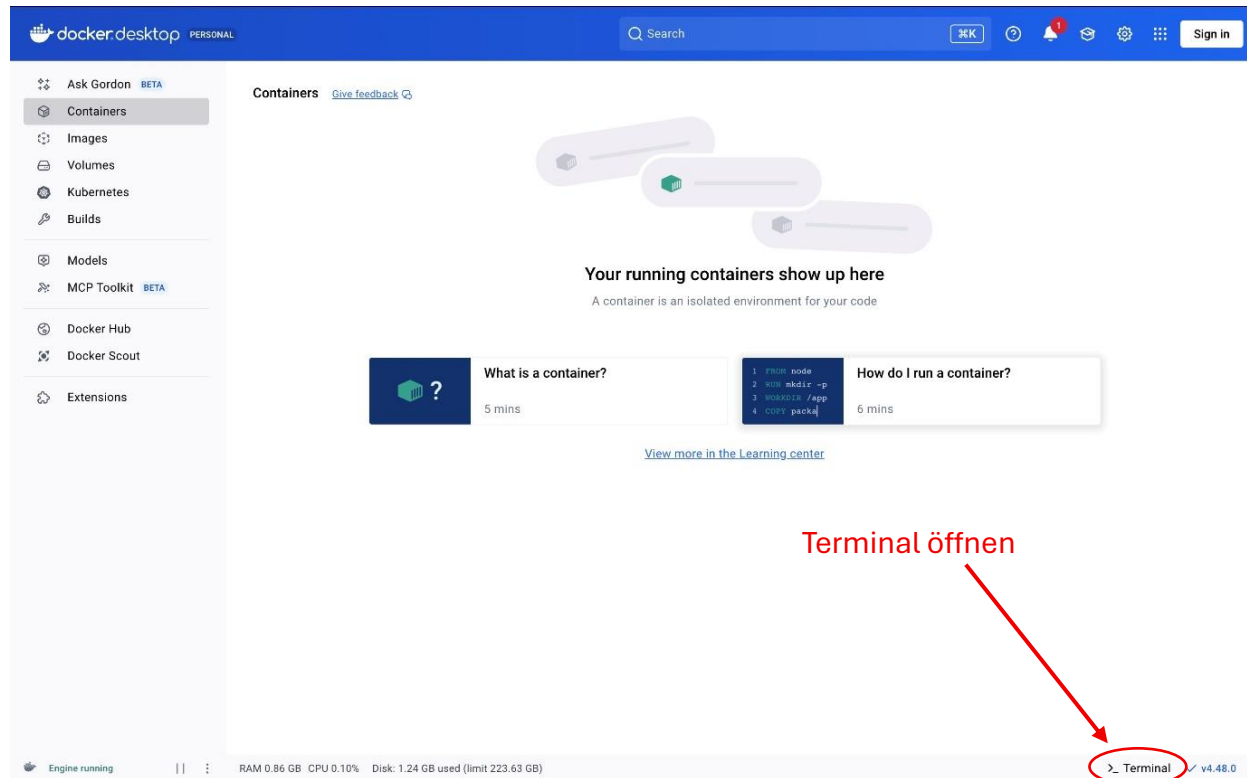
Or

  [Create an account](#) 

Zu Beginn reicht es theoretisch, die Aufforderung mit dem Skip-Button rechts oben zu überspringen, da du auch ohne ein Konto lokal Container ausführen kannst. Es ist allerdings empfehlenswert bereits ein Konto zu erstellen, weil du dieses früher oder später sowieso brauchen wirst um Images aus der offiziellen Bibliothek „Docker Hub“ zu ziehen oder um eigene Images in private Repositories hochzuladen.

3.1 Dein erster Container: Webserver nginx

Öffne nun ein Terminal, um deinen ersten Docker-Container zu starten. Das geht entweder mit dem Button **>_ Terminal** ganz unten rechts in Docker Desktop, oder indem du ein Terminal deines Betriebssystems startest.



Zuerst wollen wir einen einfachen Container starten, in dem die Anwendung „nginx“ läuft, dabei handelt es sich um einen Webserver. Führe dafür im Terminal deiner Wahl folgenden Befehl aus:

```
docker run -d -p 8080:80 nginx
```

Dieser Befehl setzt sich aus folgenden Teilen zusammen:

- ***docker run*** startet einen Docker-Container basierend auf einem Docker-Image.
- ***-d*** startet den Container im detached-Mode, das heißt dass dein Terminal nicht vom Container blockiert wird und du weitere Befehle im Terminal eingeben kannst.
- ***-p 8080:80*** definiert die Port-Weiterleitung. Der Webserver im Container ist unter dem Port 80 erreichbar, da das die Standardeinstellung bei nginx ist. Da der Container isoliert vom System ist und seine eigenen Ports hat, kannst du allerdings gar nicht auf den Port 80 des Containers zugreifen. Deshalb nutzt du diesen Befehl, um den Port 80 des Containers auf den Port 8080 deines lokalen Systems weiterzuleiten – auf diesen Port kannst du nun problemlos zugreifen und somit den Webserver nutzen.
- ***nginx*** ist der Name des Docker-Images. Ein Image ist eine Vorlage für einen Container, aus welchem der Container letztendlich gestartet wird. Wir benutzen hier die „fertige“ Vorlage nginx, welche von Docker bereitgestellt wird. Später

werden wir auch lernen, wie man eingene Docker-Images aus sogenannten Docker-Files erstellt.

Öffne nun in einem Browser deiner Wahl die Adresse <http://localhost:8080/>. Dort kannst du jetzt die Startseite deines Webserver bewundern, welcher isoliert in einem Docker-Container läuft. Diesen Webserver nun nach deinen Vorstellungen zu gestalten ist allerdings noch etwas komplexer – dafür brauchst du zunächst erstmal die Anwendungsdateien deiner tatsächlichen Webanwendung. Weiterhin benötigst du ein Docker-File, in welchem du definierst, wie man aus diesen Dateien ein Docker-Image baut, um damit dann einen Container zu starten. Tutorials zu solchen weiterführenden Themen findest du im Abschnitt 3.3.

3.2 Nützliche Befehle

Führe in der Konsole folgenden Befehl aus:

```
docker ps
```

Dieser Befehl zeigt alle aktiven Container an, im Moment sollte dort nur der Container mit deinem Webserver zu finden sein. Der Befehl ist später aber sehr nützlich, vor allem wenn du mit mehreren Containern gleichzeitig arbeitest und dir einen Überblick darüber verschaffen willst, welche Container gerade laufen und welche Ports sie benutzen. Kopiere die Container-ID deines nginx-Containers und führe folgenden Befehl aus:

```
docker stop <Container-ID>
```

Dieser Befehl stoppt deinen Container. Wenn du nun im Browser versuchst <http://localhost:8080/> aufzurufen wird die Verbindung fehlschlagen, weil der Container, in dem sich der Webserver befindet, nicht mehr läuft.

Pausierte Container kann man allerdings problemlos wieder starten oder auch für immer löschen, diese und weitere nützliche Befehle findest du in dieser Übersicht:

Befehl	Beschreibung
<code>docker ps -a</code>	Liefert Übersicht über alle aktiven und inaktiven (gestoppten) Container
<code>docker rm <ID></code>	Löscht einen Docker-Container
<code>docker start <ID></code>	Startet einen gestoppten Container erneut
<code>docker exec -it <ID> bash</code>	Öffnet eine Shell innerhalb eines laufenden Containers, ermöglicht somit Zugriff auf sein Dateisystem und seine Prozesse
<code>docker logs <ID></code>	Liefert die logs eines Docker-Containers
<code>docker images</code>	Liefert Übersicht über heruntergeladene Images
<code>docker rmi <ID></code>	Löscht ein Docker-Image
<code>docker pull <image></code>	Lädt ein vorgefertigtes Image aus Docker-Hub herunter

Hinweis: Die GUI von Docker Desktop ist zu Beginn sehr nützlich, um sich einen Überblick über lokale Images und Container zu verschaffen und diese einfach zu starten und stoppen. Auf lange Sicht ist es aber unerlässlich, sich mit den konkreten Befehlen auseinander zu setzen, weil in Anwendungsfällen oder Produktionsumgebungen oft keine GUI zur Verfügung steht.

3.3 Nächste Schritte

Docker ist sehr ein sehr mächtiges Werkzeug, welches die Bereitstellung und Verwaltung von Anwendungen stark vereinfacht und aus der Industrie nicht mehr wegzudenken ist. Allerdings mag es am Anfang auch sehr komplex und einschüchternd wirken, und man weiß gar nicht so richtig wo man anfangen soll. Deshalb haben wir hier noch ein paar Tutorials aufgelistet, welche den Einstieg mit Docker und die nächsten Schritte erklären.

Es ist absolut essentiell, zuerst ein Verständnis für das Konzept und die Grundbegriffe von Docker (Dockerfile, Image, Container, Volume) zu entwickeln. Das mag zwar langweiliger erscheinen als direkt alles praktisch auszuprobieren, wird euch aber eine Menge Zeit und Frust sparen. (Wir sprechen aus Erfahrung 😊)

- „Docker Explained in 4 Minutes“ (gutes Einstiegsvideo): [Youtube-Video](#) [4 min]
- Überblick über die Grundbegriffe von Docker: [Youtube-Video](#) [12 min]
- Sehr ausführlicher, hervorragender Docker-Crashkurs: [Youtube-Video](#) [1h]
- Eigene Anwendung (Frontend) Containerisieren, Dockerfile selbst schreiben: [Geschriebener Guide auf docker.com](#) [30 min]
- PostgreSQL-Datenbank in Docker, Docker Volumes: [Youtube-Video](#) [12 min]
- Orchestrierung von Containern mit Docker Compose: [Youtube-Video](#) [33 min]
- Ausführliche, deutschsprachige Tutorial-Reihe: [Youtube Playlist](#) [40 x 5 min]
- Offizielle Guides für diverse gängige Sprachen/Frameworks: [Auflistung](#) [je 30 min]